

Decision-OS V13 Research Note v0.1

Compound Loop: Selecting the Variable That Makes the Next Iteration 1.01

Shinichi Nagata

Research Note v0.1

Date: 2026-06-09 JST

Abstract

This research note introduces the initial concept of Decision-OS V13: Compound Loop.

Recent discussions of AI loops often focus on task completion: an agent executes, checks, revises, and repeats until the task is finished. This note distinguishes that kind of loop from a different structure: a loop that improves the next loop itself.

The core question of V13 is not merely "Can an AI repeat the task?" but "Which variable should be updated so that the next iteration becomes 1.01 instead of 1.00?"

V13 defines Compound Loop as an Aspire-bounded, Carrier-preserving, positive-EV update loop. Its purpose is to select the variable whose improvement increases the expected value of future iterations without increasing ruin risk, damaging Aspire, or destroying the Carrier that must continue the trajectory.

The note separates Task Loop, Evolution Loop, and Compound Loop, and positions V13 as the layer after V10 and V12: V10 recalculates survivable goal-length, V12 prevents false completion and preserves restartability, and V13 governs which variable may be updated so that repeated operation compounds safely.

Core Claim:

A loop is valuable not because it repeats action, but because it raises the expected value of the next iteration without breaking Aspire.

1. Motivation

AI systems are increasingly described in terms of loops. An agent can plan, execute, observe, revise, and repeat. This is useful, but incomplete.

A task loop can improve completion. It can make an agent better at finishing a defined job. However, completion alone does not explain long-term compounding. A system may repeat tasks while still repeating the same structural mistake. It may optimize effort while increasing ruin risk. It may accelerate a negative-EV process.

The central problem is therefore not only how to loop, but what the loop learns.

If the wrong variable is updated, repeated action becomes degradation. If the right variable is updated under safe constraints, repeated action becomes compound improvement.

V13 begins from this question:

After one loop completes, what should be updated so that the next loop is slightly better?

2. Three Loop Types

2.1 Task Loop

A Task Loop repeats actions inside a fixed objective.

Typical form:

1. Execute.
2. Check.
3. Fix.
4. Repeat until complete.

The Task Loop asks:

Did the system finish the task?

This is useful for coding, writing, testing, automation, and ordinary agent workflows.

However, a Task Loop may leave the deeper system unchanged. The next loop may not become better. The agent may complete the task while preserving the

same weak observation points, same fragile assumptions, same cost structure, or same hidden risk.

2.2 Evolution Loop

An Evolution Loop updates the system itself.

It asks:

What changed after the task, and what should the system learn?

This is broader than task completion. It may update procedure, memory, evaluation, constraints, routing, or future planning.

However, Evolution Loop is still dangerous if it updates too broadly or too aggressively. Not every failure means the objective function should change. Not every success means the system should scale.

2.3 Compound Loop

A Compound Loop is a controlled update loop that selects the variable whose improvement makes the next iteration stronger.

It asks:

Which variable should be improved so that the next loop becomes 1.01 instead of 1.00?

This is the V13 layer.

The Compound Loop does not simply repeat action. It improves the future condition from which action is repeated.

3. Definition: Compound Loop

A Compound Loop is an Aspire-bounded update process that:

1. observes the result of a prior iteration,
2. identifies the variable that most improves the next iteration,
3. updates that variable within ruin-risk limits,
4. preserves the Carrier of Aspire,
5. repeats only when the game remains positive-EV.

Minimal form:

Compound Loop = Positive-EV detection + Variable selection + Ruin-risk cap + Aspire preservation + Iteration increase.

The loop is not valuable because it moves faster. It is valuable because it improves the condition of the next move.

4. Positive-EV Loop Principle

V13 inherits a practical probability principle:

Participate in positive games.

Increase iteration when the structure is positive-EV.

Do not increase iteration when the process is negative-EV.

The danger of AI agents is that they can amplify both positive and negative games.

A positive-EV process becomes stronger when repeated under bounded risk.

A negative-EV process becomes more destructive when automated.

Therefore, before increasing iteration count, V13 asks:

Is this a positive game?

Is ruin risk bounded?

Does the loop preserve Aspire and Carrier?

Does the next iteration become safer or stronger?

If yes, iteration may increase.

If no, the loop must be capped, delayed, redirected, or blocked.

5. Aspire-Bounded Objective Update

V13 does not reject objective-function updates. It restricts them.

Objective functions may be updated only within Aspire boundaries.

An objective-function update must not:

- damage Aspire,
- destroy the Carrier,
- increase ruin risk beyond the declared cap,
- erase reversibility,

- convert known incompleteness into irreversible commitment,
- or force expansion that survives only under success assumptions.

In short:

Objective-function updates must be Aspire-bounded.

A profitable expansion that destroys the person, organization, trust, recovery capacity, or long-term direction carrying the Aspire is not evolution. It is pre-ruin acceleration.

6. Carrier and Ruin-Risk Cap

V13 distinguishes two kinds of expansion.

6.1 Breakthrough Expansion

Breakthrough Expansion occurs when a system increases scope, spending, automation, or risk while preserving the Carrier.

The expansion may be difficult, but failure remains recoverable.

6.2 Pre-Ruin Expansion

Pre-Ruin Expansion occurs when success would look like a breakthrough, but failure would destroy the Carrier.

Examples include:

- subscription spending without recovery path,
- business expansion that assumes immediate revenue,
- trading size that ignores bankruptcy probability,
- automation that scales a negative-EV process,
- public release with known irreversible defects,
- or growth that consumes health, credibility, sleep, relationships, or decision capacity.

V13 rejects this type of expansion.

The rule is:

Ruin probability cannot be ignored merely because the upside is large.

7. Controlled Territory Principle

V13 favors controlled adjacent expansion over blind central breakthrough.

The intuition is strategic.

In territory games, it may be safer to take the outer region first rather than attack the capital directly. In shogi, taking an ordinary pawn may be more valuable than chasing a flashy tactic if the pawn improves the future position and reduces risk.

The same applies to AI operation.

A good AI loop should not jump directly toward the largest visible return. It should expand into controllable adjacent territory:

- low downside,
- observable result,
- reversible action,
- next-step improvement,
- reduced future risk,
- preserved Carrier.

This produces the Controlled Territory Principle:

A good loop does not maximize the next move.

It improves the position from which future moves become safer.

8. Small Edge Compounding

In human games, a small disadvantage can sometimes be reversed by a brilliant move.

In AI-assisted environments, both sides may be able to search strong moves. This reduces the probability that a weaker position is naturally repaired by opponent error.

Therefore, early small advantages matter more.

A small positive edge can compound through:

- more attempts,
- better data,

- lower error rate,
- more trust,
- more capital,
- more time,
- better optionality,
- and safer future moves.

This gives the Small Edge Compounding principle:

When both sides can search strong moves, early small advantages compound rather than disappear.

The goal of V13 is not to find a single divine move. It is to preserve and compound small positive edges.

9. Known Incompleteness Rule

Loops require exploration. Some information cannot be known before acting.

However, V13 distinguishes unknowns from known defects.

Unknowns can justify exploration.

Known defects cannot justify irreversible expansion.

Known incompleteness may proceed only if it is:

- bounded,
- observable,
- reversible,
- capped,
- and connected to a learning purpose.

Known incompleteness must not be converted into irreversible commitment.

This is especially important in AI operation because AI can run experiments cheaply. If small tests are available, blind jumps become less justified.

AI loops should reduce the need for blind jumps.

10. V10 / V12 / V13 Layering

V13 sits after V10 and V12.

V10: Survival-Bounded Planning

V10 asks:

Is the current goal-length still survivable?

If the original goal-length begins to consume the Carrier of Aspire, the trajectory must be rescaled.

V12: Completion Integrity

V12 asks:

Is this actually complete in a way that the future self or next agent can restart?

It prevents false completion, bad handoff, and context-loss collapse.

V13: Compound Loop

V13 asks:

After completion or partial completion, which variable should be updated so that the next loop improves?

Thus:

V10 protects the trajectory from fixed-goal collapse.

V12 protects completion from false closure.

V13 protects iteration from non-compounding repetition.

11. Operational Gate

V13 may use a four-outcome gate:

GO

The loop is positive-EV, ruin risk is bounded, and the next iteration is likely to improve.

HOLD

The loop may be positive, but the sign, cost, recovery path, or Carrier impact is unclear. Observe before scaling.

CAP

The loop may proceed only under a fixed cap: money, time, exposure, automation authority, trading size, publication scope, or iteration count.

BLOCK

The loop is negative-EV, ruin-risk exposed, Aspire-damaging, Carrier-damaging, or dependent on irreversible success assumptions.

CAP is especially important in V13 because many real decisions are neither full GO nor full BLOCK. They are valid only under bounded exposure.

12. Core Claim

A loop is valuable not because it repeats action.

A loop is valuable when it selects the variable that makes the next iteration stronger without increasing ruin risk or breaking Aspire.

In one line:

V13 is the discipline of turning 1.00 into 1.01 without destroying the Carrier that must keep compounding.

13. Final Statement

Task loops finish work.

Evolution loops update systems.

Compound loops select the variable that makes the next loop better.

Decision-OS V13 defines Compound Loop as an Aspire-bounded, Carrier-preserving, positive-EV update process. Its purpose is not to maximize the next move, but to improve the position from which future moves become safer, stronger, and more repeatable.

The practical rule is simple:

Before increasing iteration, check the sign of the game.

If the loop is positive-EV and ruin risk is bounded, increase iteration.

If the loop is negative-EV or Carrier-damaging, automation only accelerates collapse.

The next frontier of AI loops is therefore not merely autonomy.

It is compounding without self-destruction.